

Ev-SLAM: Event-based Visual SLAM Through Adaptive Neural Radiance Fields

Haoyu Li

Department of Computer Science
University of Pittsburgh
Pittsburgh, USA
hal308@pitt.edu

Jingao Xu

School of Computer Science
Carnegie Mellon University
Pittsburgh, USA
jingaox@andrew.cmu.edu

Longfei Shangguan

Department of Computer Science
University of Pittsburgh
Pittsburgh, USA
longfei@pitt.edu

Abstract—Event cameras are increasingly integrated into SLAM systems to mitigate motion blur and enhance tracking performance in low-light environments. However, the event stream by nature lacks essential semantic and color information, posing a fundamental challenge to visual feature extraction and mapping. This paper introduces **Ev-SLAM**, a novel event-based visual SLAM system that harnesses implicit neural 3D scene representation to enhance visual feature extraction and SLAM performance. We design three modules to achieve this goal, including (1) a dynamic-slicing algorithm to address the impact of polarity neutralization in event frame generation; (2) a point cloud representation that minimizes redundant sampling in free space, significantly reducing the computational overhead of neural radiance field (NeRF) training; and (3) an optical-flow-based approach for coarse camera pose and depth estimation, coupled with a scale feedback mechanism to bootstrap NeRF training. Extensive evaluations demonstrate that **Ev-SLAM** outperforms SOTA baselines Point-SLAM and Co-SLAM, achieving up to an 82.9% improvement in tracking accuracy under normal-speed conditions and 85.2% under fast-speed conditions.

I. INTRODUCTION

Visual Simultaneous Localization and Mapping (vSLAM) is the key building block of autonomous driving [1], [2], robot navigation [3], and augmented reality [4]. Conventional vSLAM systems relying on frame cameras often face challenges in highly dynamic environments or poor lighting conditions due to motion blur and extended exposure times. Recently, *event cameras* have emerged as a promising alternative capable of addressing these limitations by providing high sampling rates, exceptional low-light sensitivity, and resistance to motion blur [5]. These advantages have driven growing interest in event-based vSLAM.

However, the asynchronous nature of event cameras makes it challenging to extract spatial feature from raw event streams for feature mapping. Prior works [6], [7] address this issue by accumulating events into event frames or time surfaces, enabling the use of vision algorithms such as Arc [8] and ORB [9] for feature extraction. Yet, event frames and time surfaces still

lack semantic and color information, resulting in weaker visual features and reduced SLAM accuracy.

Recently, the advancement in Neural Radiance Field (NeRF) technology have expanded scene reconstruction beyond RGB images to event streams [7], [10], [11]. Compared to event frames or time surface representations, implicit neural 3D scene representations retain richer semantic information, which can potentially enhance visual feature extraction and matching in vSLAM.

Building on this insight, in this paper we propose an event-based vSLAM system that utilizes NeRF as its backbone, enabling simultaneous camera tracking and scene reconstruction. However, translating this idea into a real system poses the following two challenges.

- **Excessive training overhead and unknown camera pose.** NeRF requires several hours of training to achieve high-fidelity scene reconstruction [12]. For event-based NeRF, this training overhead can easily exceed ten hours due to the substantial volume of event data that needs to be processed, as summarized in Table I. This high latency makes it challenging to directly apply event-based NeRF to visual SLAM tasks, such as obstacle avoidance [13] and path planning [14]. Moreover, existing event-based NeRF algorithms [6], [7], [10], [11] all rely on precise camera pose information for scene reconstruction. However, in SLAM systems, the camera pose itself is an unknown parameter to be estimated.

- **Polarity neutralization.** Event cameras encode brightness changes using two polarity symbols: +1 for increases and -1 for decreases [5]. To integrate event data into NeRF for scene reconstruction, existing event-based NeRF algorithms [6], [7] accumulate events over a fixed time interval to generate an event frame. However, accumulating events over a fixed time interval can cause severe polarity neutralization. Polarity neutralization effectively cancels out the information about brightness changes, leading to a loss of critical detail in the accumulated event data. Figure 1 shows an example where crucial geometric information get lost due to polarization neutralization. This loss can significantly impair the

TABLE I: Comparison of exiting event NeRFs [15]. Ev-SLAM employs online optimization to process incoming events, iteratively performing mapping and tracking. It eliminates the need for extensive offline NeRF training.

Methods	Event Representation	Offline Training	Training Time
E-NeRF [10]	Event Pair	✓	12 hours
Robust e-NeRF [11]	Event Pair	✓	23 hours
EventNeRF [7]	Event Frame	✓	12 hours
Ours	Event Frame	✗	—

ability to reconstruct high-quality images or accurately estimate the scene’s geometry, particularly in dynamic environments where fine-grained temporal information is crucial.

To address these challenges, we leverage event-based optical flow to estimate coarse camera poses and depth, with the predicted depth constrained by a scale feedback mechanism. We then propose an adaptive event frame generation method that monitors neutralized pixels and event count, thereby mitigating polarity neutralization while preserving high-quality geometric information. To further facilitate tracking and mapping, we employ a neural point cloud representation to reduce redundant sampling in free space. The logarithmic brightness change computed from the event stream is used to supervise training. Once tracking is completed, we remap the scene to maintain accurate geometric structure and compute the appropriate scale for incoming events.

We implement this idea by building Ev-SLAM and evaluate it on real world SLAM dataset. The comprehensive evaluation shows that Ev-SLAM outperforms SOTA solutions Point-SLAM [16] and Co-SLAM [17], achieving up to an 82.9% improvement in tracking accuracy under normal speed conditions and 85.2% under fast speed conditions. For rendering tasks, Ev-SLAM enhances PSNR by 14.13 dB and 11.02 dB under normal speed and fast speed conditions, respectively. In summary, our contributions are as follows:

- We present the design and implementation of Ev-SLAM, the first solution to successfully integrate event-based NeRF into SLAM systems, enhancing its pose tracking and scene reconstruction accuracy in fast speed conditions.
- We propose three simple yet very effective modules to initialize NeRF rendering without accurate pose priors; minimize its excessive computation overhead; and mitigate the impact of polarity neutralization on event frame generation.
- Our evaluations on multiple public datasets show that Ev-SLAM outperforms two SOTA baselines in both tracking accuracy and rendering performance across varying scene dynamics.

II. NERF PRIMER

We begin by introducing how neural radiance field (NeRF) works. NeRF [18] is parameterized by an MLP

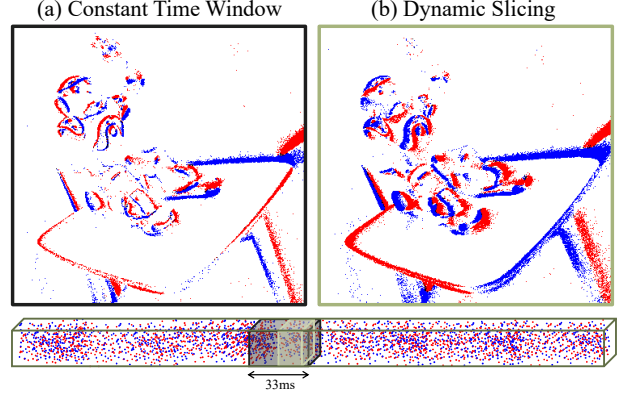


Fig. 1: (a) Prior works [7] accumulate events using a constant time window can lead to significant loss of geometric information due to polarity neutralization. (b) Ev-SLAM adapts the slicing window based on scene dynamics, effectively preserving geometric details for improved feature extraction and mapping.

$\Phi_{\Theta} : (\mathbf{x}, \mathbf{d}) \mapsto (\mathbf{c}, \sigma)$, which maps input coordinates $\mathbf{x} \in \mathbb{R}^3$ and viewing direction $\mathbf{d} \in \mathbb{S}^2$ to the predicted color $\mathbf{c} \in \mathbb{R}^3$ and volume density $\sigma \in \mathbb{R}$. The predicted color $\hat{\mathbf{C}}(\mathbf{u})$ at pixel $\mathbf{u} \in \mathbb{R}^2$ is calculated by volumetric rendering [19] along a camera ray $\mathbf{r}$ with N points using the following equations:

$$\hat{\mathbf{C}}(\mathbf{u}) = \sum_{i=1}^N T_i \alpha_i \mathbf{c}_i, \quad T_i = \exp \left(- \sum_{j=1}^{i-1} \sigma_j \delta_j \right) \quad (1)$$

where, T_i characterizes the accumulated transmittance along the ray and δ_j denotes the interval between consecutive samples, while $\alpha_i = 1 - \exp(-\sigma_i \delta_i)$ represents the opacity from alpha compositing at point i .

To conduct volumetric rendering, NeRF samples entire 3D space through ray marching, *which usually leads to long reconstruction delay due to the slow per-scene optimization and the unnecessary sampling of empty space*. This nature introduces a significant challenge to adapt NeRFs as a **SLAM** system.

Event-based NeRF. Unlike RGB cameras, event cameras capture brightness changes asynchronously rather than producing intensity images. Event stream $\{E_k\}_{k=1}^{N_{evs}}$ consist of N_{evs} events $e_k = (\mathbf{u}_k, p_k, t_k)$, where $\mathbf{u}_k \in \mathbb{R}^2$ denotes the pixel coordinate with polarity $p_k \in \{-1, +1\}$ at timestamp t_k . To adapt the event stream into a 2D intensity frame format, prior works [6], [7] utilize event frames that accumulate events at each pixel over a *constant time interval* as input for training NeRFs. The original NeRF framework optimizes a least squares error between the predicted color $\hat{\mathbf{C}}$ and the ground truth color $\mathbf{C}$. In contrast, event-based NeRFs are supervised by matching the brightness change ΔL_{t_k} :

$$\Delta L_{t_k} = L(\mathbf{u}_k, t_k) - L(\mathbf{u}_k, t_{k-1}) = p_k C_p \quad (2)$$

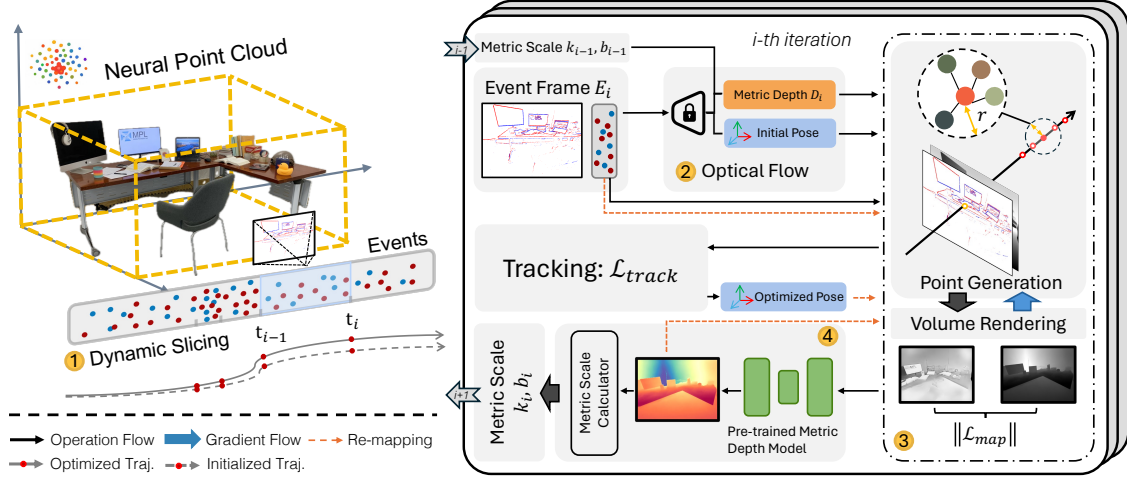


Fig. 2: Overview of Ev-SLAM. Detailed explanation of system workflow can be found in §III.

where L is the logarithmic brightness and C_p represents the contrast threshold. By querying the MLP $\mathbf{F}_\Theta$ and performing volumetric rendering, the predicted brightness change $\Delta \hat{L}_{t_k}$ can be obtained for model training.

III. SYSTEM DESIGN

Ev-SLAM operates in four stages, as illustrated in Figure 2. ①: it first slices the event stream based on scene dynamics to generate event frames, effectively reducing the impact of polarity neutralization and enhancing neural rendering performance for SLAM optimization. ②: Ev-SLAM then utilizes optical flow to estimate coarse camera poses and depth, providing essential inputs for NeRF. ③: Ev-SLAM adaptively generates neural points based on event density, allowing the point cloud to dynamically adjust to the scene’s complexity. For rendering, we focus on point sampling around objects, minimizing redundant sampling in free space. ④: Finally, a pre-trained depth estimator is employed to refine the depth scale, enabling depth correction for incoming frames. We show the algorithm sketch in Algorithm 1 and elaborate on each module in the following sections.

A. Event Frame Generation Through Adaptive Slicing

We slice the event stream and aggregate all events within each segment i into an event frame E_i . However, an insufficient number of events within each slice would result in poor optical flow estimation for system initialization (detailed in §III-B), whereas an excessively long slicing window increases the occurrence of polarity-neutralized pixels, leading to loss of geometric information and degraded tracking accuracy. Prior works [6], [7] use a constant slicing interval fail to achieve consistently high performance under different scene dynamics.

To address this limitation, we propose to adapt the slicing window to scene dynamics based on the following two key criteria: (1) the current number of events exceeds φ , or (2) the proportion of neutralized events,

N_{neu} , reaches a predefined threshold ρ . The first criterion adapts to scene dynamics, ensuring responsiveness to motion/scene changes, while the second preserves fine structural details by preventing excessive event neutralization. We set φ and ρ to 7500 and 3%, which empirically optimize the performance. As illustrated in Figure 1, this adaptive approach dynamically adjusts the window length based on motion speed – expanding for slow movement and contracting for rapid camera motion, while effectively retaining local scene details.

B. Cold Start: Coarse Pose and Depth Estimation

The generated event frames are then feed into NeRF for scene reconstruction. NeRF requires known camera poses to accurately reconstruct the scene. However, in SLAM systems, the camera pose is unknown and must be estimated during the process. To address this chicken-egg problem, we propose using optical flow to derive a coarse initial estimate of both the camera pose and scene depth for initialization. As new event frames are added while the SLAM robot is moving, we use these new event frames to refine the pose estimates progressively, improving the accuracy of both the reconstruction and pose tracking process.

(i) **Coarse Pose Estimation.** For an event frame E_i , we first compute pixel-level correspondences between frame E_i and its neighboring frames using a pretrained optical flow estimator [20]. Leveraging these correspondences, we estimate the relative camera pose by solving the essential matrix [21]. The pose estimated from optical flow corresponds to the motion between two successive camera views, represented by a rotation matrix $\tilde{R}_i^r$ and a translation matrix $\tilde{T}_i^r$. Given the absolute camera pose at time slot $i-1$, denoted by R_{i-1}^a, T_{i-1}^a , the absolute camera pose at time slot i , R_i^a, T_i^a , can be computed as:

$$\begin{aligned} R_i^a &= R_{i-1}^a \cdot \tilde{R}_i^r, \\ T_i^a &= T_{i-1}^a + R_{i-1}^a \cdot \tilde{T}_i^r \end{aligned} \quad (3)$$

This coarse camera pose subsequently serves as the initial bootstrap for training the NeRF model.

(ii) Monocular Depth Estimation. For an event frame E_i , the optical flow also estimates the depth D_i^f for each event. However, such monocular depth information lacks physical scale in unbounded scenes, which leads to pronounced scale disparity.

To address this issue, we leverage the physical scale k_{i-1} and depth bias b_{i-1} , obtained from processing the previous event frame E_{i-1} , as described in the next section. Since the slicing window of each event frame spans only a few to tens of milliseconds, we can reasonably assume that variations in the physical depth scale are negligible. Therefore, the scaled depth information, denoted as D_i , can be estimated as follows:

$$D_i = k_{i-1} D_i^f + b_{i-1} \quad (4)$$

where the depth scale factor k_i and depth bias b_i can be estimated by minimizing the discrepancy between the metric depth D_i^m and the depth D_i^f predicted by the optical flow using the following equation:

$$\min_{k_i, b_i} \sum \left(k_i D_i^f + b_i - D_i^m \right)^2 \quad (5)$$

In the above equation, the metric depth D_i^m is obtained by using a pretrained metric depth estimator [26] after obtaining the intensity image I_i through volumetric rendering, as described in Section III-C2. In this way, we can obtain a coarse pose and depth estimates serve as the initial geometric priors for scene reconstruction. These parameters will be gradually refined through joint NeRF optimization during tracking and mapping.

C. Online Rendering Through Point Cloud Representation and Adaptive Sampling

Conventional NeRF employs global MLP to implicitly model scene radiance via ray marching. However, this implicit representation results in long reconstruction delays due to the slow per-scene network optimization and the unnecessary sampling of large regions of empty space. For instance, following NeRF’s default sampling rate of $N = 192$ points per ray [18], rendering a single high quality (1080p) image entails around 400 million network calls, posing a significant computational burden.

To reduce computational overhead while maintaining rich scene representation, we replace the MLP representation in NeRF with a neural point cloud representation:

$$P = \{(q_i, f_i^b, f_i^g) | i = 1, \dots, N\} \quad (6)$$

where each point i , located at position $q_i \in \mathbb{R}^3$, is associated with a brightness feature f_i^b and a geometric feature f_i^g , which are derived from the Event to Image Translation (EIT) and Event to End-task Learning (EEL) branches, respectively [27]. The 3D position q_i of each

Algorithm 1: Ev-SLAM Online Optimization

Input: Event Stream $\{M\}$
Output: Neural Points P and Poses $\{R_i, T_i\}$

```

1 Initialization;
2 while  $M \geq 0$  do
3   /* Event Frame: Sec. III-A */
4    $E_i \leftarrow \text{DynamicSlicing}()$ ;
5   /* Cold Start: Sec. III-B */
6    $\{R_i^c, T_i^c\}, D_i^f \leftarrow \text{OpticalFlow}(E_i)$ ;
7    $D_i \leftarrow \text{DepthCorrection}(D_i^f, k_{i-1}, b_{i-1})$ ;
8   /* Rendering: Sec. III-C2 */
9    $\text{Mapping}(E_i, D_i, \{R_i^c, T_i^c\})$ ;
10   $\{R_i, T_i\} \leftarrow \text{Tracking}(E_i, D_i)$ ;
11   $I_i \leftarrow \text{Mapping}(E_i, D_i, \{R_i, T_i\})$ ;
12   $(k_i, b_i) \leftarrow \text{ScaleOptimization}()$ ;
13   $M \leftarrow M - E_i$ ;
14   $i \leftarrow i + 1$ ;

```

point is obtained using the depth estimates introduced in §III-B. By doing so, we obtain a set of spatially distributed neural points that approximate the underlying scene geometry. This approach eliminates the need for ray sampling in empty regions, enabling adaptive sampling and more efficient updates.

Next, we explain how to generate the point cloud and render the scene as the camera moves.

1) *Point Cloud Generation:* For each incoming event frame, we back-project pixel events into world coordinates using depth information estimated from optical flow. Around each projected point, we run a local search within a sphere of radius r to identify nearby existing points that may aid in accurate scene reconstruction. If no neighboring points are detected within this range, we generate five new neural points along the viewing ray. These points are centered at depth D and are distributed over interval $[(1-\alpha)D, (1+\alpha)D]$, where α is set to 0.01 in this paper. If neighboring points are found, no new points are added. This strategy helps reduce redundant neural points and ensures computational efficiency, as demonstrated by our experiments.

2) *Rendering:* Building on the previously generated neural point cloud, we render images from new viewpoints using volumetric rendering. Since the points are generated around the surface of objects, we sample them between $[(1-\alpha)D, (1+\alpha)D]$. Specifically, given a camera pose with origin $\mathbf{o}$ and orientation $\mathbf{d}$, we sample points x_i along the ray as follows:

$$x_i = \mathbf{o} + z_i \mathbf{d}, \quad i \in \{1, \dots, S\}, \quad (7)$$

where $z_i \in \mathbb{R}$ is the point depth; S denotes the number of points sampled within the depth range. Inspired by Point-SLAM [16], we use a MLP decoder to estimate occupancies o_i and colors c_i for each point and perform volumetric rendering to obtain brightness intensity $\hat{I}(\mathbf{u}_k)$ and depth $\hat{D}(\mathbf{u}_k)$ at pixel $\mathbf{u}_k$. The features of each 3D

TABLE II: Comparison of Ev-SLAM and baseline systems on **Vector** [22] dataset using **ATE RMSE[cm]** metric.

Method	robot		sofa		desk		hdr	
	normal	fast	normal	fast	normal	fast	normal	fast
EVO [23]	13.04	–	–	–	–	–	–	–
ESVO [24]	–	–	–	–	53.01	–	–	–
E2VID [25]+Co-SLAM [17]	10.21	158.5	41.18	157.8	27.36	97.13	79.22	86.95
E2VID [25]+Point-SLAM [16]	15.87	28.10	37.59	64.36	22.47	65.01	41.78	48.18
Ev-SLAM (Ours)	8.94	23.40	21.71	27.84	29.10	31.17	13.58	14.75

coordinate x_i can be then obtained through a Gaussian positional encoding [28].

Event Rendering. The rendered brightness intensity is a weighted average of the brightness along each ray:

$$\hat{I}(\mathbf{u}_k) = \sum_{i=1}^N \alpha_i \mathbf{c}_i, \quad \alpha_i = \sigma_i \prod_{j=1}^{i-1} (1 - \sigma_j), \quad (8)$$

where α_i denotes the weight associated with each depth value, determined by the opacity σ_i . The logarithmic brightness can be calculated as follows:

$$\begin{aligned} L(\mathbf{u}_k, t_k) &= \text{linlog}(I(\mathbf{u}_k)) \\ &= \begin{cases} I(\mathbf{u}_k) \cdot \ln(B)/B, & \text{if } I(\mathbf{u}_k) < B \\ \ln(I(\mathbf{u}_k)), & \text{else.} \end{cases} \end{aligned} \quad (9)$$

In line with [29], we set $B = 20$ for better modeling events distribution in real world. The logarithmic brightness change is computed using Eq. 2, with the input event stream serving as the supervision signal. We discuss the training process in the next section.

Geometry Rendering. The rendered depth at a given pixel is computed using the following weighted sum:

$$\hat{D}(\mathbf{u}_k) = \sum_{i=1}^M \alpha_i z_i, \quad (10)$$

where z_i represents the depth value along the viewing ray. The product term accounts for the accumulated transmittance along the ray, ensuring that the contribution of each depth sample is appropriately weighted.

D. Mapping and Tracking

Building upon the event-based and geometry rendering modules, we leverage the input event stream and the estimated depth as supervision signals. In the following, we describe how our system utilizes them to perform robust mapping and tracking.

1) *Mapping:* During mapping, we render two consecutive intensity images, $\hat{I}_{i-1}$ and $\hat{I}_i$, to compute the logarithmic brightness change $\Delta \hat{\mathbf{L}}_i$ as:

$$\Delta \hat{\mathbf{L}}_i = \text{linlog}(\hat{I}_i) - \text{linlog}(\hat{I}_{i-1}) \quad (11)$$

We minimize the rendering loss between the estimated brightness change $\Delta \hat{\mathbf{L}}_i$ and the observed event measurement $\Delta \mathbf{L}_i$. The mapping loss is thus defined as:

$$\mathcal{L}_{\text{map}} = \left| D_i - \hat{D}_i \right|_2 + \lambda_b \left\| \Delta \mathbf{L}_i - \Delta \hat{\mathbf{L}}_i \right\|_2^2, \quad (12)$$

where D_i represents the estimated depth value and $\Delta \mathbf{L}_i = C\{p_k\}_{e_k \in E_i}$ is measured by event camera, with λ_b as a hyperparameter controlling the balance between the two terms.

2) *Tracking:* To refine the pose estimation, we minimize the tracking loss $\mathcal{L}_{\text{track}}$, which consists of two components: a geometric loss and a brightness difference loss. These are combined with a weighting factor λ_t to balance their contributions:

$$\mathcal{L}_{\text{track}} = \left| D_i - \hat{D}_i \right|_2 + \lambda_t \left\| \Delta \mathbf{L}_i - \Delta \hat{\mathbf{L}}_i \right\|_2^2 \quad (13)$$

The first term of the loss function penalizes discrepancies in depth estimation, ensuring accurate geometric alignment. The second term measures the difference in brightness changes, enforcing consistency in the visual appearance between frames.

IV. EXPERIMENTS

A. Experimental Setups

Ev-SLAM is implemented in Python and trained on a PC server equipped with an NVIDIA RTX 6000 Ada GPU. To the best of our knowledge, there is currently no publicly available event-only dense vSLAM pipeline suitable for direct comparison. Therefore, we compare Ev-SLAM with existing state-of-the-art RGB-D NeRF-based SLAM methods, using frame reconstruction via E2VID [25]. Specifically, we first employ E2VID to reconstruct grayscale images from event data, which are then used as inputs to Co-SLAM [17] and Point-SLAM [16]. We also compare Ev-SLAM with event-based visual odometry methods. Specifically, we include EVO [23] and ESVO [24] as representative baselines. Ev-SLAM is evaluated on the multi-sensor SLAM dataset Vector [22]. We measure localization accuracy using the Absolute Trajectory Error (ATE) in centimeters (cm). To quantitatively evaluate rendering quality, we

TABLE III: Rendering quality comparison across four scenes under normal and fast motion conditions on the Vector dataset [22]. We report PSNR ($\uparrow$), SSIM ($\uparrow$), and LPIPS ($\downarrow$) to evaluate rendering quality on multiple scenes.

Method	Metric	robot		sofa		desk		hdr	
		normal	fast	normal	fast	normal	fast	normal	fast
E2VID [25]+Point-SLAM [16]	PSNR $\uparrow$	26.88	25.11	15.12	10.57	20.72	8.120	12.50	11.25
	SSIM $\uparrow$	0.833	0.857	0.511	0.504	0.819	0.359	0.373	0.421
	LPIPS $\downarrow$	0.248	0.466	0.872	0.618	0.421	0.526	0.587	0.666
Ev-SLAM (Ours)	PSNR $\uparrow$	28.84	25.92	24.16	17.11	19.34	18.62	26.63	22.27
	SSIM $\uparrow$	0.864	0.833	0.807	0.725	0.769	0.717	0.845	0.739
	LPIPS $\downarrow$	0.240	0.219	0.471	0.379	0.479	0.506	0.231	0.283

adopt Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM) [30], and Learned Perceptual Image Patch Similarity (LPIPS) [31] with AlexNet as the backbone.

B. Tracking Performance

We report the tracking performance on the Vector dataset [22] in Table II. On average, Ev-SLAM significantly outperforms existing event-based visual odometry methods and RGB-D baselines using E2VID inputs. It achieves the best accuracy across most scenarios, including both normal and fast motions, as well as high dynamic range (HDR) conditions. In the challenging `robot_fast` sequence, Ev-SLAM reduces the ATE by 82.9% compared to E2VID [25]+Co-SLAM [17].

We attribute this improvement to the use of neural point cloud representation and logarithmic brightness supervision, which together ensure more accurate geometric reconstruction and richer gradient signals during training. In contrast, event-based visual odometry methods often struggle under complex scenes and rapid motion, frequently failing to complete tracking. RGB-D SLAM baselines are highly sensitive to aggressive motion, suffering from significant performance degradation.

C. Rendering Performance

We compare the rendering performance in Table III, where Ev-SLAM consistently outperforms the baseline E2VID+Point-SLAM [16] in terms of PSNR, SSIM, and LPIPS. Notably, under challenging scenarios such as `sofa-fast` and `hdr-fast`, Ev-SLAM improves PSNR by 6.54 dB and 1.02 dB respectively, while also achieving much better perceptual quality, as shown in Figure 3.

These improvements indicate that our neural point cloud representation, combined with photometric supervision from high-temporal-resolution event data, enables more accurate and perceptually faithful rendering. As shown in Figure 3, Ev-SLAM maintains sharp details and structure even under fast motion and low-light conditions, where the baseline suffers from blur and ghosting. In the `desk_fast` scenario, due to overexposure, the ground truth grayscale holdout frames fail to reveal the

TABLE IV: Runtime on Vector `robot_normal`.

Method	Tracking		Mapping	
	/Iteration	/Frame	/Iteration	/Frame
Point-SLAM [16]	27 ms	1.17 s	34 ms	11.46 s
Ev-SLAM (ours)	34 ms	1.40 s	51 ms	19.82 s

text on the screen. Ev-SLAM is still able to reconstruct the text on the screen. Moreover, in dynamic scenes, Ev-SLAM effectively eliminates motion blur, producing sharp and temporally consistent images, which further demonstrates the advantage of using event streams for high-quality rendering.

D. System Latency

As shown in Table IV, Ev-SLAM takes 34 ms per iteration for tracking and 51 ms for mapping. Despite this moderate increase in runtime (only a few milliseconds per iteration) compared against the baseline, our method delivers significant performance improvement in both tracking accuracy and mapping completeness. This trade-off is particularly worthwhile in scenarios where robustness and high-quality reconstruction are critical.

Recall from Table I that existing event-based NeRF approaches [7], [10], [11] require 12 to 23 hours of offline training. In contrast, Ev-SLAM performs joint mapping and tracking in an online optimization manner, making it the first event-based NeRF system successfully adapted for online SLAM, closing the gap between high-fidelity event-based neural rendering and practical visual odometry. Our design demonstrates that with neural point cloud representation and event-frame supervision, we can avoid the need for time-consuming offline training while still achieving competitive runtime results.

E. Effectiveness of Dynamic Slicing

We now evaluate the impact of slicing strategies across various sequences. Table V presents a detailed comparison of PSNR results across different slicing strategies and scenes. Our dynamic slicing method consistently achieves the best performance in most scenarios, as indicated by the highlighted top scores. Notably, in certain scenes such as `robot_fast` and `desk_normal`,

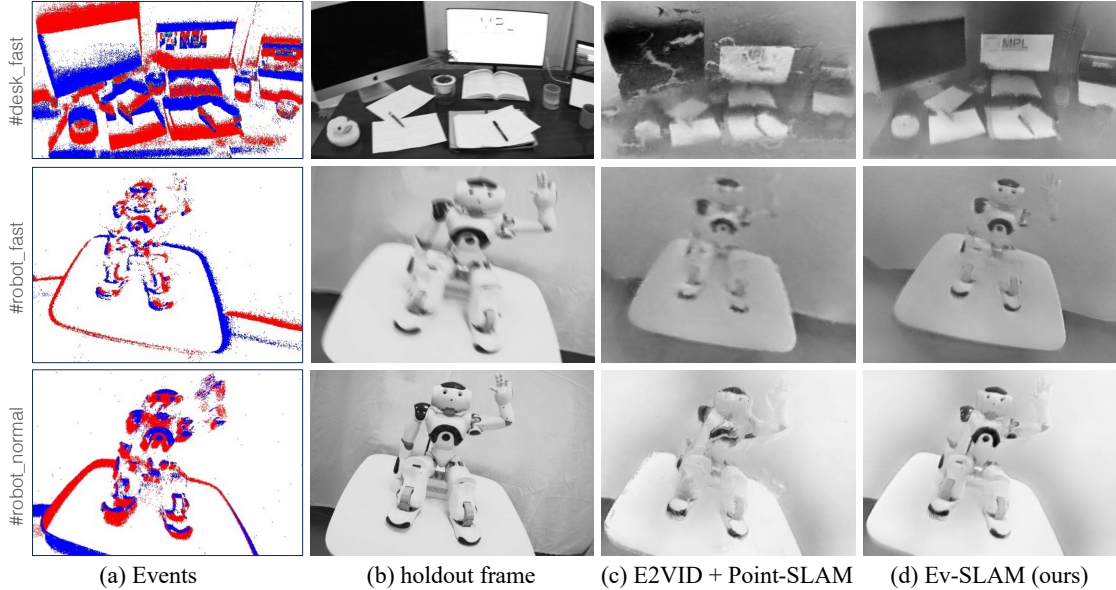


Fig. 3: Rendering results. (a) Event frame visualization. (b) Holdout frames that are excluded from training, used for visual comparison. (c) The E2VID + Point-SLAM baseline struggles with motion blur and fails to recover structure details. (d) Ev-SLAM reconstructs accurately under challenging conditions such as overexposure and rapid motion.

TABLE V: Evaluation of slicing strategies in terms of PSNR ($\uparrow$) across multiple scenes.

Window	robot		sofa		desk		hdr	
	normal	fast	normal	fast	normal	fast	normal	fast
10 ms	16.56	23.20	15.11	16.82	17.11	18.21	18.14	18.02
30 ms	27.19	24.69	22.37	16.39	20.03	18.25	20.82	20.04
50 ms	27.99	21.11	23.56	14.71	18.92	17.06	23.95	21.77
100 ms	20.55	17.30	19.83	13.86	17.23	15.29	26.21	21.67
200 ms	18.12	16.95	18.66	12.45	16.07	14.99	24.46	19.79
Ours	28.84	25.92	24.16	17.11	19.34	18.62	26.63	22.27

some fixed slicing windows (e.g., 30 ms or 50 ms) produce results that closely approach or even slightly surpass our method. However, the optimal fixed window varies significantly across scenes and motion types. This variability actually demonstrates the advantage of our adaptive approach, which generalizes well across diverse dynamic and static conditions without relying on hand-tuned window sizes.

V. RELATED WORK

Event-based SLAM. Event-based SLAMs can be broadly classified into three categories: *feature-based*, *direct methods*, and *motion-compensation-based*.

Feature-based methods rely on hand-crafted features extracted from frame-based event representations for tracking and mapping [32], [33]. However, the performance of these methods heavily depends on the robustness of feature detection, which can degrade significantly under fast motion. Direct methods bypass explicit data association by directly aligning events in space and time to estimate camera motion and scene structure. While direct methods are generally more robust to challenging

motion and lighting conditions, they are limited to sparse or semi-dense reconstruction and have not yet been extended to support fully dense mapping.

Motion-compensation-based methods [34] estimate the camera’s motion by optimizing the alignment of asynchronous events in a motion-compensated frame. Despite their effectiveness in specific motion scenarios, these approaches often suffer from the *event collapse* problem – when events accumulated over long temporal windows converge into degenerate structures such as lines or points, resulting in a loss of spatial information.

In contrast to these approaches, our work introduces a novel direction by generating a 3D point cloud directly from event data and employing a neural radiance field for dense map reconstruction and camera tracking. This enables a unified framework capable of exploiting both the high temporal resolution of event streams and the expressive power of implicit scene representations.

Event-based Neural Radiance Field. Ev-NeRF [6] is the first to attempt training NeRFs using event data under typical camera motion speeds and general illumination conditions. EventNeRF [7] utilizes a color event camera combined with a rotational turntable to reconstruct objects. The goal is to achieve dense and photorealistic scene reconstruction in a static environment. E-NeRF [10] combines the closest events for each pixel to reconstruct a neural radiance field from a fast-moving event camera. Robust e-NeRF [11] proposes a more realistic event generation model with two complementary normalized loss for high-fidelity reconstruction from noisy event streams. However, all these methods require accurate camera poses as inputs for novel view synthesis.

Δ_t NeRF [35] further introduces an event camera tracker that minimizes the error between sparse events and the temporal gradient of the scene representation on simplified intensity-change events. Nevertheless, this method requires a pre-trained NeRF as prior.

VI. CONCLUSION

This paper presents **Ev-SLAM**, the first SLAM system that integrates an event-based neural radiance field into an online SLAM pipeline. **Ev-SLAM** comprises three key components: a dynamic slicing algorithm that mitigates polarity neutralization, an optical-flow-based initialization for bootstrapping NeRF training, and a point cloud representation that facilitates efficient NeRF optimization. Unlike existing event-based NeRF systems, **Ev-SLAM** operates fully online, progressively rendering the scene while simultaneously refining pose estimation as the SLAM robot moves, eliminating the need for an extended offline NeRF training process. Experimental results demonstrate that **Ev-SLAM** surpasses baseline methods in both tracking and rendering accuracy under a range of challenging conditions.

VII. ACKNOWLEDGMENTS

We thank all reviewers for their constructive feedback. This material is based on work supported by National Science Foundation (NSF) under Grant No.2302724.

REFERENCES

- [1] G. Chen, H. Cao, J. Conradt, H. Tang, F. Rohrbein, and A. Knoll, "Event-based neuromorphic vision for autonomous driving: A paradigm shift for bio-inspired visual sensing and perception," *IEEE Signal Processing Magazine*, 2020.
- [2] A. I. Maqueda, A. Loquercio, G. Gallego, N. García, and D. Scaramuzza, "Event-based vision meets deep learning on steering prediction for self-driving cars," in *Proc. of CVPR*, 2018.
- [3] B. He, Z. Wang, Y. Zhou, J. Chen, C. D. Singh, H. Li, Y. Gao, S. Shen, K. Wang, Y. Cao *et al.*, "Microsaccade-inspired event camera for robotics," *Science Robotics*, 2024.
- [4] P. R. Desai, P. N. Desai, K. D. Ajmera, and K. Mehta, "A review paper on oculus rift-a virtual reality headset," *arXiv*, 2014.
- [5] G. Gallego, T. Delbrück, G. Orchard, C. Bartolozzi, B. Taba, A. Censi, S. Leutenegger, A. J. Davison, J. Conradt, K. Daniilidis *et al.*, "Event-based vision: A survey," *IEEE transactions on pattern analysis and machine intelligence*, 2020.
- [6] I. Hwang, J. Kim, and Y. M. Kim, "Ev-nerf: Event based neural radiance field," *arXiv* 2023.
- [7] V. Rudnev, M. Elgharib, C. Theobalt, and V. Golyanik, "Event-nerf: Neural radiance fields from a single colour event camera," *arXiv*, 2022.
- [8] I. Alzugaray, "Event-driven feature detection and tracking for visual slam," Ph.D. dissertation, ETH Zurich, 2022.
- [9] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "Orb-slam3: An accurate open-source library for visual, visual-inertial, and multimap slam," *IEEE transactions on robotics*, 2021.
- [10] S. Klenk, L. Koestler, D. Scaramuzza, and D. Cremers, "E-nerf: Neural radiance fields from a moving event camera," *arXiv*, 2023.
- [11] W. F. Low and G. H. Lee, "Robust e-nerf: Nerf from sparse & noisy events under non-uniform motion," *arXiv*, 2023.
- [12] K. Gao, Y. Gao, H. He, D. Lu, L. Xu, and J. Li, "Nerf: Neural radiance field in 3d vision, a comprehensive review," *arXiv*, 2022.
- [13] J. Xu, D. Li, Z. Yang, Y. Zhao, H. Cao, Y. Liu, and L. Shang-guan, "Taming event cameras with bio-inspired architecture and algorithm: A case for drone obstacle avoidance," in *Proceedings of the ACM MobiCom*, 2023.
- [14] T. M. Cabreira, L. B. Brisolara, and F. J. Paulo R, "Survey on coverage path planning with unmanned aerial vehicles," *Drones*, 2019.
- [15] J. Huang, C. Dong, and P. Liu, "Inceventgs: Pose-free gaussian splatting from a single event camera," *arXiv*, 2024.
- [16] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald, "Point-slam: Dense neural point cloud-based slam," in *Proc. of ICCV*, 2023.
- [17] H. Wang, J. Wang, and L. Agapito, "Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam," in *Proc. of CVPR*, 2023.
- [18] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *CoRR*, 2020.
- [19] J. T. Kajiya and B. P. Von Herzen, "Ray tracing volume densities," in *Proc. of SIGGRAPH*, 1984.
- [20] S. Shiba, Y. Klose, Y. Aoki, and G. Gallego, "Secrets of event-based optical flow, depth and ego-motion estimation by contrast maximization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [21] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. of CVPR*, 2016.
- [22] L. Gao, Y. Liang, J. Yang, S. Wu, C. Wang, J. Chen, and L. Kneip, "Vector: A versatile event-centric benchmark for multi-sensor slam," *IEEE Robotics and Automation Letters*, 2022.
- [23] H. Rebecq, T. Horstschaefer, G. Gallego, and D. Scaramuzza, "Evo: A geometric approach to event-based 6-dof parallel tracking and mapping in real time," *IEEE Robotics and Automation Letters*, 2017.
- [24] Y. Zhou, G. Gallego, H. Rebecq, L. Kneip, H. Li, and D. Scaramuzza, "Semi-dense 3d reconstruction with a stereo event camera," in *Proc. of ECCV*, 2018.
- [25] H. Rebecq, R. Ranftl, V. Koltun, and D. Scaramuzza, "High speed and high dynamic range video with an event camera," *IEEE transactions on pattern analysis and machine intelligence*, 2019.
- [26] S. F. Bhat, R. Birkel, D. Wofk, P. Wonka, and M. Müller, "Zoedepth: Zero-shot transfer by combining relative and metric depth," *arXiv*, 2023.
- [27] L. Wang, Y. Chae, and K.-J. Yoon, "Dual transfer learning for event-based end-task prediction via pluggable event to image translation," in *Proc. of ICCV*, 2021.
- [28] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng, "Fourier features let networks learn high frequency functions in low dimensional domains," *arXiv*, 2020.
- [29] Y. Hu, S.-C. Liu, and T. Delbruck, "v2e: From video frames to realistic dvs events," *arXiv*, 2021.
- [30] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, 2004.
- [31] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. of CVPR*, 2018.
- [32] W. Guan, P. Chen, Y. Xie, and P. Lu, "Pl-evio: Robust monocular event-based visual inertial odometry with point and line features," *IEEE Transactions on Automation Science and Engineering*, 2023.
- [33] J. Manderscheid, A. Sironi, N. Bourdis, D. Migliore, and V. Lepetit, "Speed invariant time surface for learning to detect corner points with event-based cameras," *arXiv*, 2019.
- [34] C. Gu, E. Learned-Miller, D. Sheldon, G. Gallego, and P. Bideau, "The spatio-temporal poisson point process: A simple model for the alignment of event camera data," in *Proc. of ICCV*, 2021.
- [35] M. Masuda, Y. Sekikawa, and H. Saito, "Event-based camera tracker by ∇_t nerf," *arXiv*, 2023.